

**[SOFTWARE EDIFICIO MIRADOR]
(DAS) Documento Arquitectura de Software
Versión 1.0**

Identificación de Documento

Identificación	EM1
Proyecto	SOFTWARE EDIFICIO MIRADOR
Versión	1.3
Documento mantenido por	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
Fecha de última revisión	31/03/2026
Fecha de próxima revisión	02/04/2026
Documento aprobado por	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
30/03/2026	1.0	Empezamos el documento DAS y se hablo como grupo que se hace en el ítem 1, logramos hacer 3 contenidos	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
31/03/2026	1.1	Continuamos el documento DAS, y logramos resumir o modificar cosas anteriores para que se entienda mejor	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
31/03/2026	1.2	Terminamos el primer ITEM y continuamos con el segundo ITEM	Consuelo Betancur, Eduardo tapia, Álvaro Salazar

Tabla de Contenidos

1. INTRODUCCIÓN.....	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	5
2. VISIÓN DEL SISTEMA	9
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	9
2.2. OBJETIVOS DEL SISTEMA.....	9
2.3. REQUERIMIENTOS FUNCIONALES	9
2.4. REQUERIMIENTOS NO FUNCIONALES	10
2.5. SUPUESTOS Y DEPENDENCIAS.....	10
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	11
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	11
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS.....	12
4.1. VISTA DE ESCENARIO.....	12
4.1.1. <i>Propósito</i>	12
4.1.2. <i>Actores</i>	12
4.1.3. <i>Diagrama general de casos de uso</i>	13
4.1.4. <i>Diagrama de casos de uso específicos</i>	13
4.1.6. <i>Especificación de casos de uso</i>	14
4.2. VISTA LÓGICA.....	15
4.2.1. <i>Propósito</i>	15
4.2.2. <i>Diagrama de clases</i>	16
4.2.3. <i>Descripción diagrama de clases</i>	16
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	17
4.3.1. <i>Propósito</i>	17
4.3.2. <i>Diagrama de componente</i>	18
4.3.3. <i>Descripción diagrama de componente</i>	18
4.3.4. <i>Diagrama de paquete</i>	18
4.3.5. <i>Descripción diagrama de paquete</i>	19
4.4. VISTA DE PROCESOS	19
4.4.1. PROPÓSITO	19
4.4.2. DIAGRAMA DE ACTIVIDAD	20
4.4.3. DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD	20
4.5. VISTA FÍSICA.....	21
4.5.1. <i>Propósito</i>	21
4.5.2. <i>Diagrama de despliegue</i>	21
4.5.3. <i>Descripción diagrama de despliegue</i>	21
5. REQUISITOS DE CALIDAD	22

5.1. PROPÓSITO	22
5.3. <i>Reglas y criterios de evaluación de calidad</i>	23
6. PRINCIPIOS DE DISEÑO APLICADOS	24
6.1. <i>Propósito</i>	24
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	24
7. PROTOTIPO	24
7.1. PROPÓSITO	24
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	24
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	24
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	24
8.1. PROPÓSITO	24
8.2. LISTA DE VERIFICACIÓN	24
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	24
9. CONTROL DE VERSIONES	24
9.1. PROPÓSITO	24
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	24
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	25
7. CONCLUSIONES	25
8. BIBLIOGRAFÍA	25
9. ANEXOS	25
9.1. CARTA GANTT	25

1. INTRODUCCIÓN

1.1. Contexto del Problema

El problema se sitúa en que una comunidad de gran tamaño, ‘Edificio Mirador’, depende actualmente de un sistema de gestión obsoleto. Esta situación genera una serie de dificultades, entre las que destacan la falta de confianza por parte de los residentes, la ineficiencia en los procesos de cobro y un aumento en los niveles de morosidad, debido principalmente a la inexistencia de un mecanismo eficaz para identificar y monitorear oportunamente a quienes cumplen con sus pagos y a quienes presentan atrasos.

La raíz del problema radica en el **procesamiento manual de datos y la falta de una base de datos centralizada**, lo que impide la integridad de la información y genera cuellos de botella en la validación de cobros. Técnicamente, la inexistencia de algoritmos de cálculo automatizado para intereses y la carencia de módulos de notificación en tiempo real provocan errores humanos recurrentes, imposibilitando una gestión financiera transparente y escalable para una comunidad de gran tamaño.

1.2. Propósito

Implementar un sistema de gestión moderno y automatizado que permita mejorar el control administrativo y financiero del edificio, facilitando la toma de decisiones, el seguimiento de pagos y la gestión eficiente de la información.

1.3. Ámbito

El proyecto se centra en modernizar la administración del edificio "El Mirador", optimizando la determinación, cálculo automatizado y notificación de los montos correspondientes a gastos comunes, disminuyendo la morosidad, agilizando procesos internos y fortaleciendo la comunicación con los residentes a través del seguimiento de solicitudes y reclamos, asegurando un funcionamiento más eficiente y transparente.

1.4. Definiciones, acrónimos y abreviaciones

ACRONIMO	DESCRIPCION

1.5. Resumen ejecutivo

El Edificio Mirador padece una crisis de morosidad y desconfianza causada por un sistema de gestión manual y obsoleto que impide la integridad de los datos. Se propone implementar una solución tecnológica automatizada bajo una arquitectura

MVC, diseñada para centralizar la información y eliminar errores humanos en el cálculo de deudas.

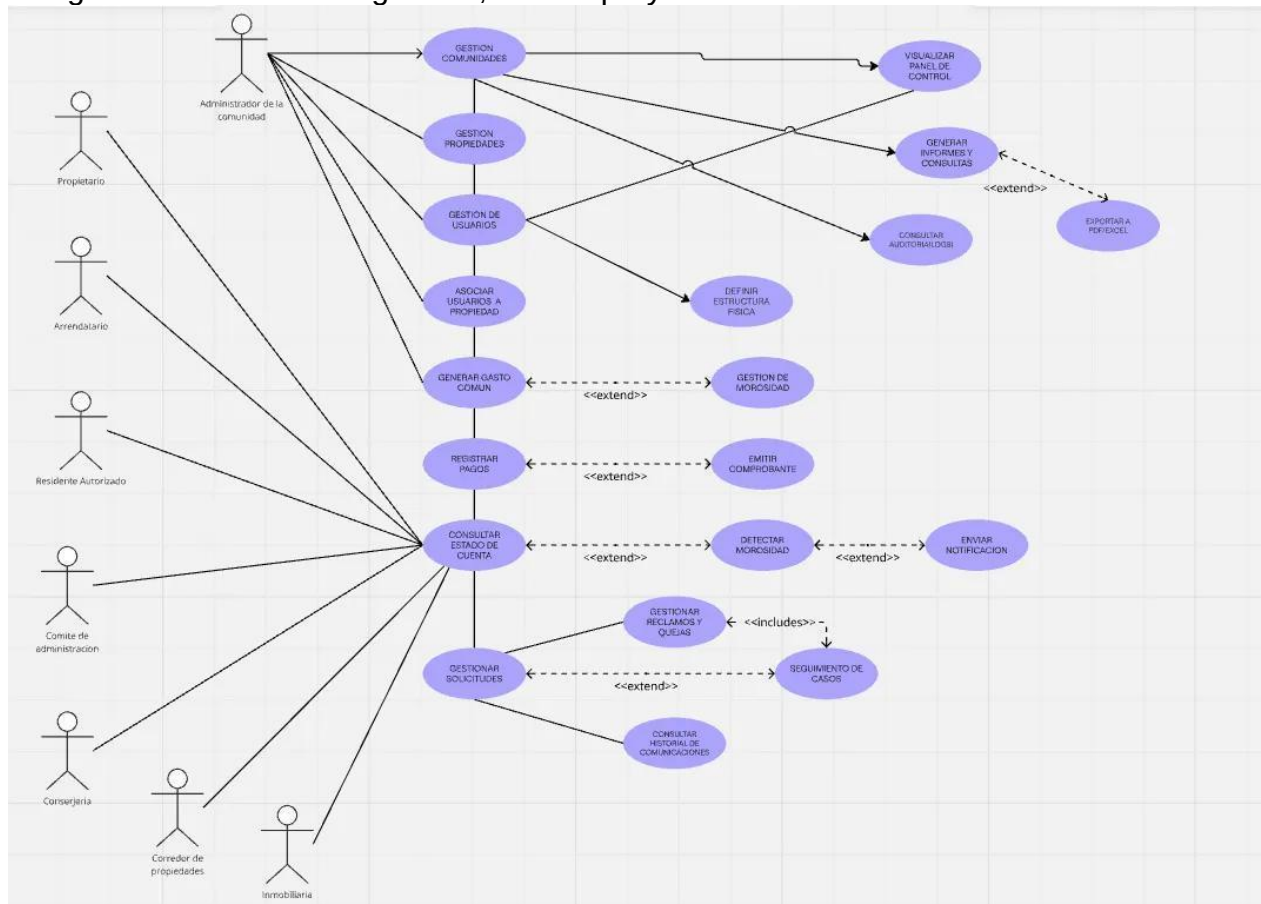
El sistema define dos perfiles críticos: el Administrador, responsable de la gestión financiera y reportes de cobranza, y el Residente, quien podrá consultar estados de cuenta y gestionar reclamos en tiempo real. Con esta modernización, se garantiza la trazabilidad total de los fondos, una comunicación interna fluida y una reducción drástica de la morosidad mediante notificaciones automáticas.

1.6. Arquitectura del sistema

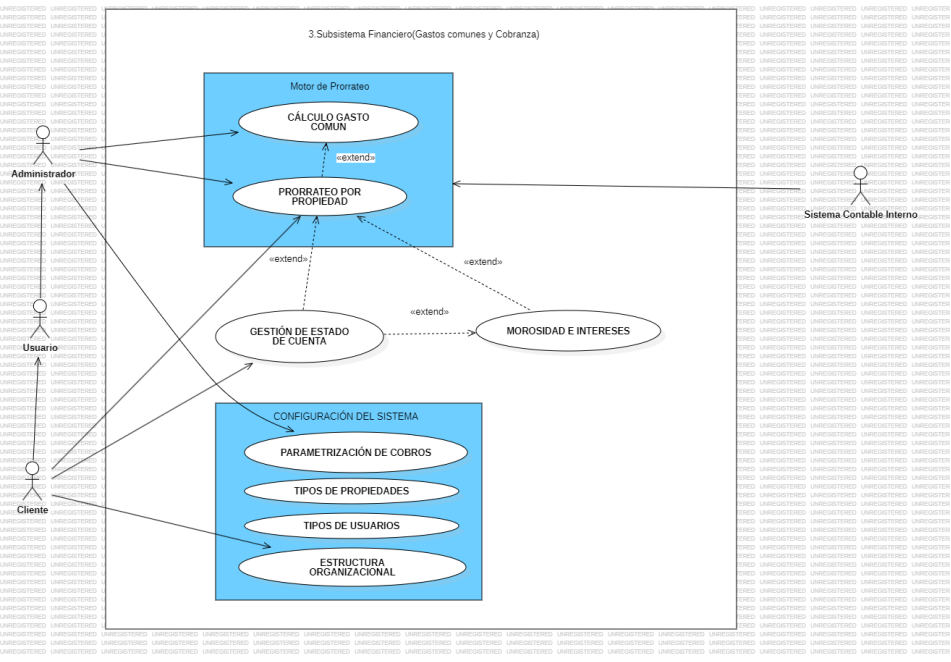
(ej. vista de escenario, vista lógica, vista de desarrollo, vista de proceso, vista física)

VISTA ESCENARIO

-Diagrama Casos de uso general, todo el proyecto

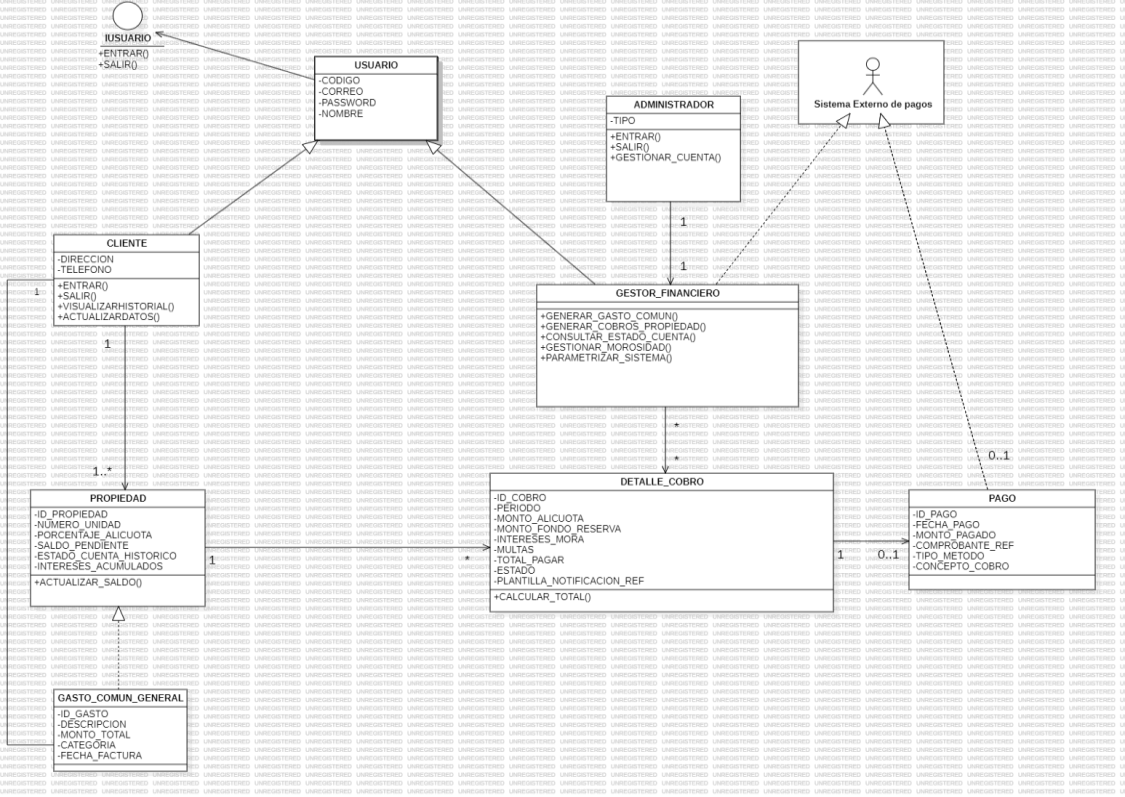


-Diagrama caso de uso subsistema 3



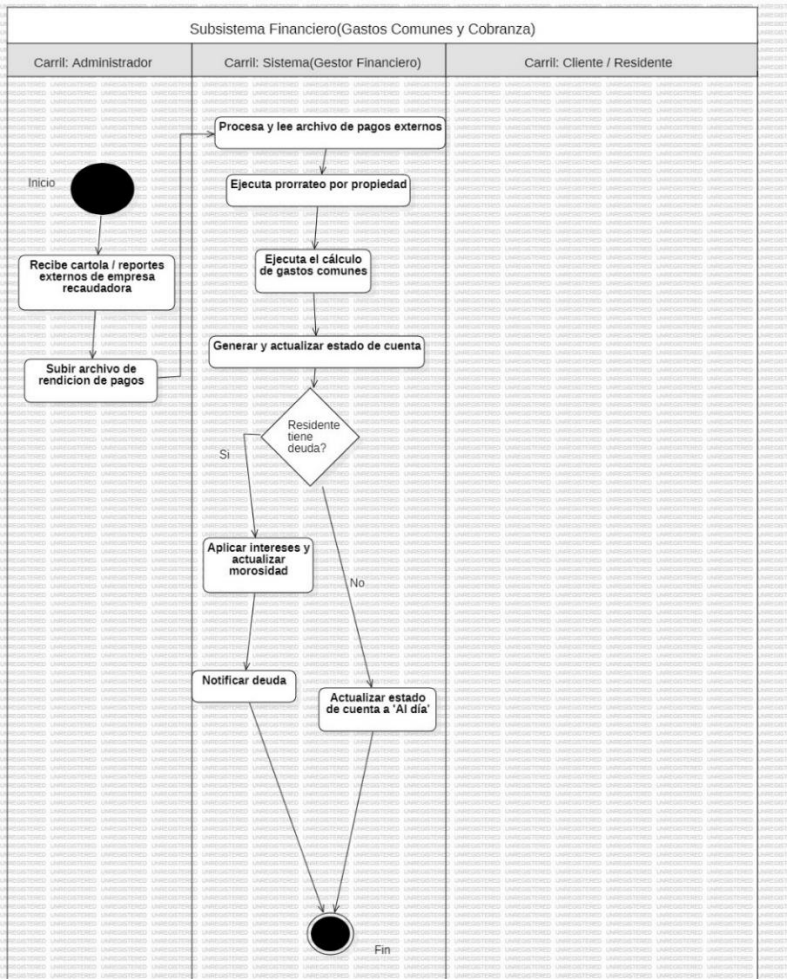
VISTA LOGICA

-Diagrama de clases



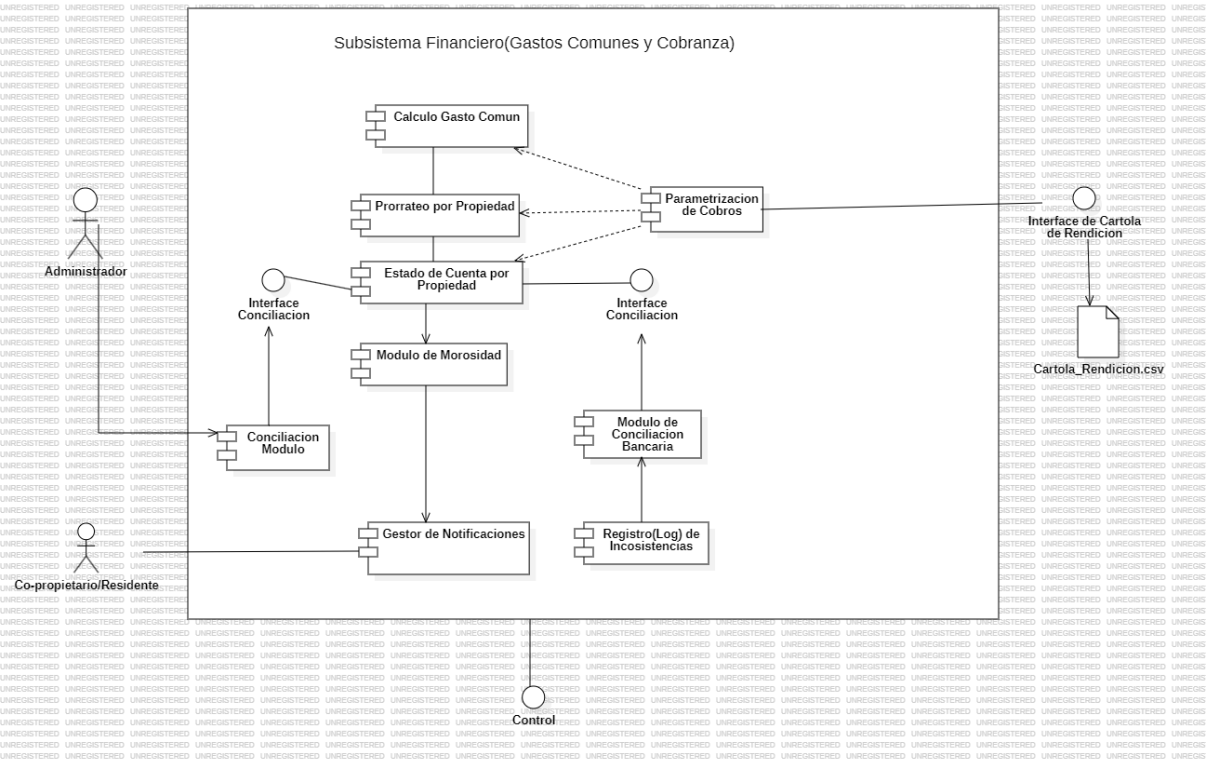
VISTA DE PROCESOS

-Diagrama de actividades

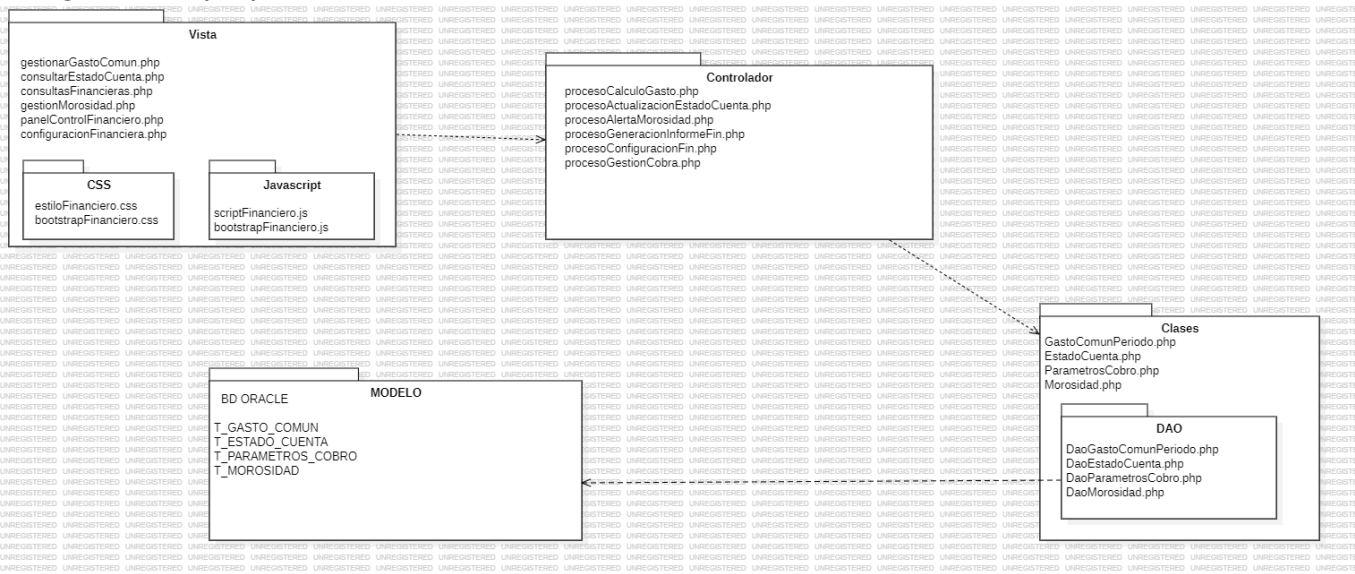


VISTA DESARROLLO

-Diagrama de componentes

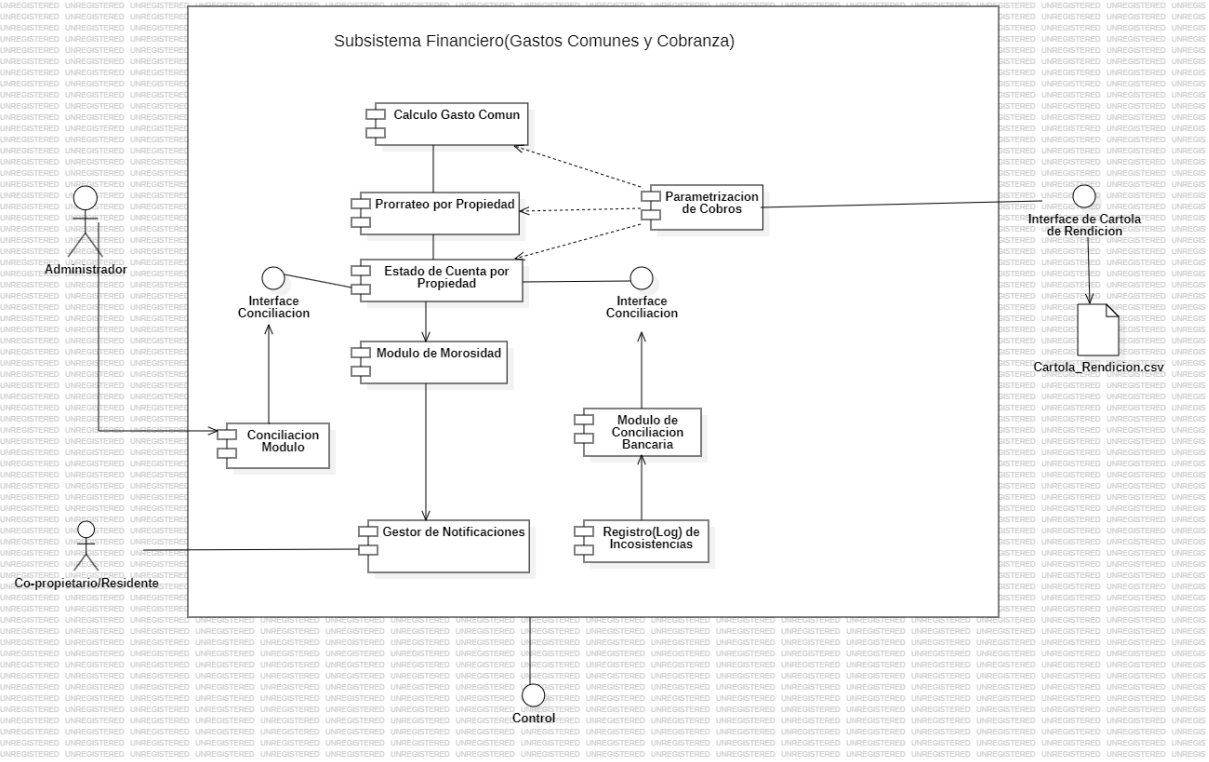


-Diagrama de paquetes



VISTA FÍSICA

-Diagrama de despliegue



2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema es una plataforma de gestión que permitirá administrar residentes, registrar pagos de gastos comunes, controlar la morosidad y generar reportes financieros, la organización y transparencia en la administración del edificio Mirador.

2.2. Objetivos del sistema

Desarrollar e implementar una solución tecnológica que permita automatizar el cálculo del gasto común, el prorrateo por propiedad y el control de morosidad del edificio "Mirador", integrando la carga de datos externos de recaudación para asegurar la trazabilidad del estado financiero de cada departamento.

2.3. Requerimientos funcionales

ID	Nombre Requerimiento	Descripción	Actor
RF-1	Gestión de residentes	Registrar, actualizar y eliminar información de residentes, asociándolos a departamentos específicos	Administrador

RF-2	Gestión de pagos de gastos comunes	Registrar pagos realizados por los residentes, Generar estados de cuenta individuales, Calcular automáticamente deudas y morosidad. Emitir comprobantes de pago	Administrador
RF-3	Gestión de Solicitudes y Reclamos	Un módulo para que el residente ingrese quejas y la administración responda	Sistema
RF-4	Sistema de Notificaciones	Automatización de correos electrónicos para avisos de cobro y alertas de morosidad	Residente / Administrador
RF-5	Control de morosidad	Identificar residentes con pagos pendientes. Generar alertas y notificaciones automáticas. Aplicar intereses o recargos por atraso (si corresponde)	Sistema
RF-6	Generación de informes	Generar reportes financieros (ingresos, deudas, morosidad).	Administrador

2.4. Requerimientos no funcionales

Usabilidad: El sistema debe ser intuitivo tanto para los residentes como para la administración del edificio.

Rendimiento y Disponibilidad: El sistema debe permitir un acceso transparente y a tiempo real de la información dentro del sistema (pagos, etc). Se sugiere que el tiempo de carga del sistema sea inferior a 2 segundos.

Fiabilidad: El sistema debe asegurar precisión en la cobranza y eficiencia de la gestión de recursos para garantizar la transparencia financiera.

Seguridad: El sistema debe garantizar la protección de los datos financieros tanto de los residentes como de la administración.

2.5. Supuestos y dependencias

Acceso a tecnología: La administración deberá contar con un computador y acceso a internet para poder actualizar los datos de pago y cobro en tiempo real.

Información de empresa: Se necesita la información de los servicios de luz, agua, etc. para poder ingresar las cifras al sistema y poder gestionar los pagos de manera automática

Capacitación: El sistema depende de que el personal esté dispuesto a capacitarse para usar el software de la manera indicada.

Comunicación: La administración es la encargada de gestionar los datos, la información se le hará llegar a los residentes directamente a su departamento o por correo electrónico si así lo desean.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado (ej. monolítico, microservicios, SOA, capas)

Para el desarrollo del Subsistema Financiero (Gastos Comunes y Cobranza) se utilizó una arquitectura en capas basada en el patrón MVC y componentes DAO.

- Capa de Presentación (Vista): Interfaces en PHP, CSS y JavaScript donde el administrador ingresa los gastos comunes (RF-08) y los residentes consultan su estado de cuenta (RF-12).
- Capa de Lógica de Negocio (Controlador): Procesa los cálculos del sistema, como el prorrateo por propiedad (RF-09), la detección de morosidad e intereses (RF-13) y las reglas de cobro configurables (RF-28).
- Capa de Acceso a Datos (Modelo / DAO): Contiene las clases del sistema y los DAO que realizan consultas SQL seguras.
- Capa de Persistencia: Base de datos Oracle, donde se almacenan las tablas financieras del sistema.

Esta arquitectura permite organizar el sistema, mejorar la seguridad de los datos y facilitar el mantenimiento.

3.2. Justificación del estilo según el contexto del sistema

Se eligió una **arquitectura en capas basada en el patrón MVC** porque permite organizar de mejor forma el **Subsistema Financiero**, encargado del cálculo y gestión de los gastos comunes.

Este estilo facilita la **mantenibilidad y escalabilidad del sistema**, permitiendo manejar correctamente procesos como la **generación del gasto común (RF-08)**, el **prorrateo por propiedad (RF-09)**, la **consulta de estado de cuenta (RF-12)** y la **gestión de morosidad e intereses (RF-13)**.

Además, permite **aislar la base de datos**, asegurando la integridad y seguridad de la información financiera configurada mediante la **parametrización de cobros (RF-28)**.

3.3. Patrones de diseño aplicados (ej. patrón MVC, repositorio, etc.)

Patrón MVC (Modelo – Vista – Controlador)

Este patrón permite separar las responsabilidades dentro del Subsistema Financiero:

- La Vista (PHP) muestra las interfaces donde el administrador ingresa los

- gastos comunes y los residentes consultan su estado de cuenta.
- El Controlador procesa la lógica del sistema, como el cálculo del prorrateo, la generación de cargos y la detección de morosidad.
 - El Modelo gestiona las entidades financieras del sistema, como gastos comunes, estados de cuenta y pagos.

Patrón DAO

El patrón DAO centraliza todas las consultas SQL hacia la base de datos Oracle, utilizadas para almacenar y recuperar información del subsistema financiero.

Esto evita que la lógica de negocio acceda directamente a las tablas, lo que mejora la seguridad, organización y mantenimiento del sistema, especialmente en operaciones relacionadas con gastos comunes, pagos y deudas.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

El propósito de la vista de escenario es definir el alcance funcional del sistema desde la perspectiva de los usuarios finales. Sirve para identificar qué actores interactúan con la plataforma y qué acciones (casos de uso) pueden realizar, validando que se cubran todas las necesidades de administración y finanzas de la comunidad.

4.1.2. Actores

Administrador de la comunidad

Usuario responsable de la gestión financiera del sistema. Puede **generar el gasto común del período (RF-08)**, ejecutar el **prorrateo por propiedad (RF-09)**, configurar reglas de cobro (**RF-28**) y gestionar procesos de cobranza y morosidad (**RF-13**).

Propietario / Arrendatario

Usuarios residentes que utilizan el sistema para **consultar su estado de cuenta (RF-12)**, revisar cargos, pagos y deudas asociadas a su propiedad.

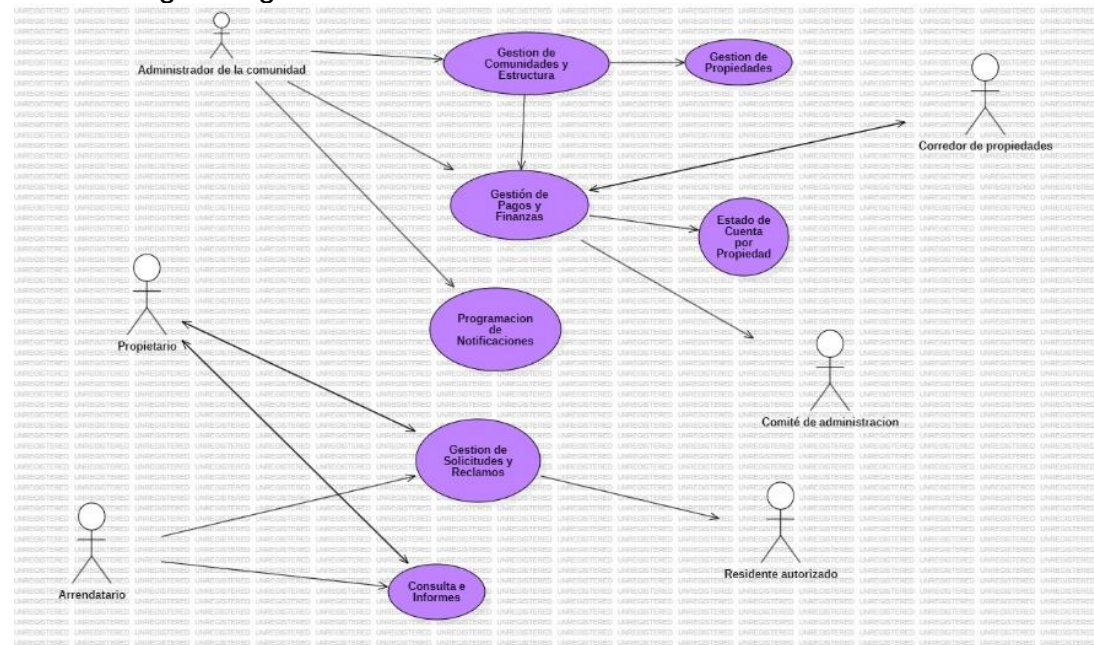
Comité de administración

Grupo supervisor que puede acceder a **información financiera e informes de gestión**, con el objetivo de supervisar el manejo de los gastos comunes y la situación de morosidad de la comunidad.

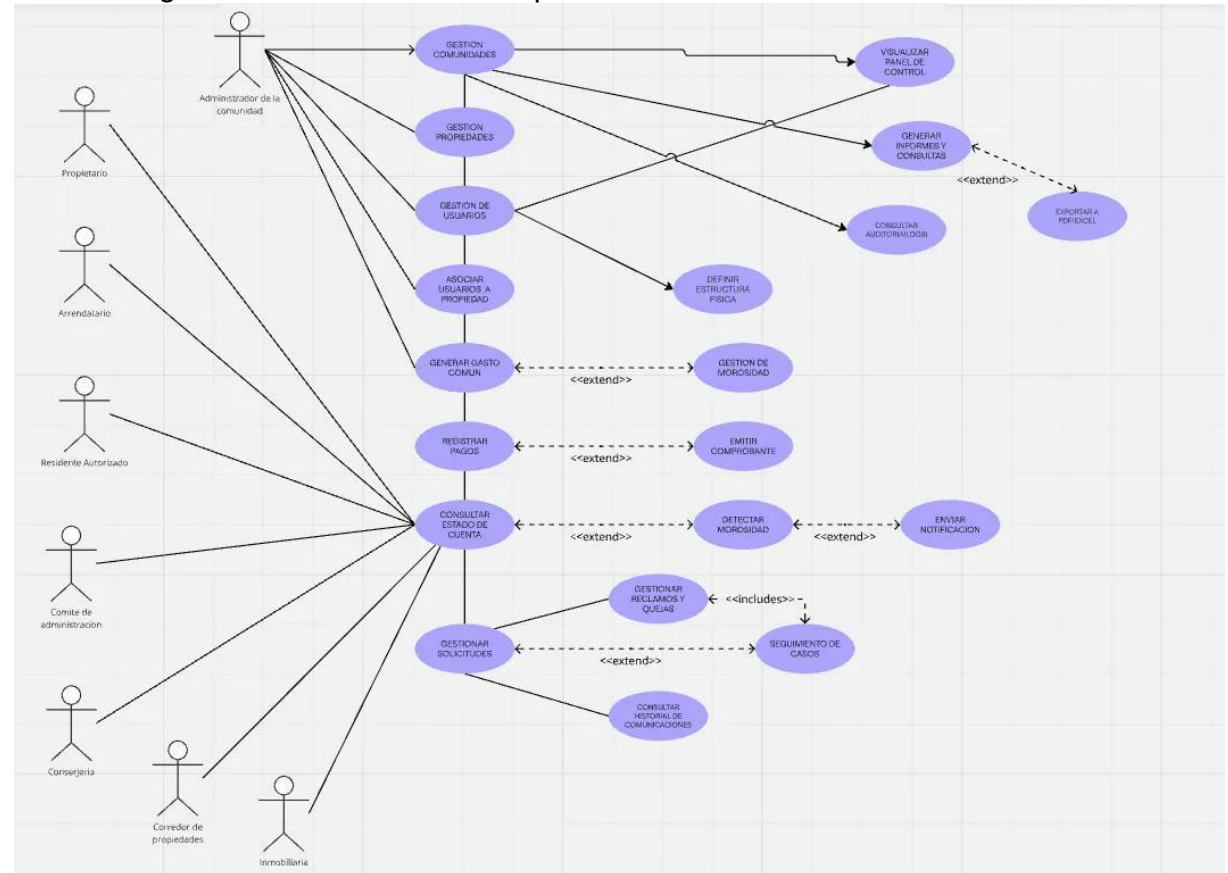
Sistemas externos de pago

Plataformas externas de pago electrónico que permiten **registrar y validar transacciones financieras** realizadas por los residentes, facilitando el proceso de recaudación del gasto común.

4.1.3. Diagrama general de casos de uso



4.1.4. Diagrama de casos de uso específicos



4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001	Registrar Gastos del Período	Administrador
CU-002	Calcular Gasto Común y Prorrateso por Propiedad	Sistema (Gestor Financiero)
CU-003	Generar y Enviar Avisos de Cobro	Sistema
CU-004	Consultar Estado de Cuenta Individual	Propietario / Residente
CU-005	Registrar o Procesar Pago de Gasto Común	Propietario / Sistema Externo de Pagos
CU-006	Detectar Morosidad y Aplicar Intereses	Sistema (Gestor Financiero)
CU-007	Registrar Pago Manual	Administrador
CU-008	Generar Estado de Cuenta por Propiedad	Sistema
CU-009	Generar Informes de Recaudación y Morosidad	Administrador

4.1.6. Especificación de casos de uso

Caso de Uso	Calcular Gasto Común y Prorrateso	Identificador: CU-003
Actores	Administrador, Sistema (Gestor Financiero).	
Tipo	Primario	
Referencias	RF-08 (Generación de gasto común), RF-09 (Generación de gasto común por propiedad), RF-12 (Estado de cuenta por propiedad)	
Precondición	El administrador ha registrado en el sistema los gastos del período correspondientes a la comunidad.	
Postcondición	El sistema calcula automáticamente el gasto común y genera los estados de cuenta individuales para cada propiedad , aplicando el prorrateso correspondiente.	
Descripción	Este caso de uso permite calcular automáticamente el monto de gasto común que corresponde a cada propiedad, distribuyendo proporcionalmente los gastos registrados del período según los criterios de prorrateso definidos por la administración..	
Resumen	Automatiza el cálculo y distribución del gasto común para cada propiedad, reduciendo errores manuales y asegurando transparencia en la gestión financiera de la comunidad.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Propietario / Residente	Ingresa al Panel de Control Financiero y selecciona la opción "Calcular Gasto Común" .
2	Sistema	Recupera y consolida los gastos registrados del período correspondientes a la comunidad (RF-08) .
3	Sistema	Ejecuta el proceso de prorrateo por propiedad, distribuyendo los costos según los criterios definidos en el sistema (RF-09) .
4	Sistema	Consulta la base de datos para actualizar los estados de cuenta de cada propiedad , incorporando los nuevos cargos del período (RF-12) .
5	Sistema	Verifica si existen deudas pendientes o propiedades morosas y calcula los intereses correspondientes según la configuración del sistema (RF-13 y RF-28) .
6	Sistema	Genera el estado de cuenta individual para cada propiedad y lo deja disponible para su consulta en la plataforma.

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
2a	Gastos del período no registrados: Si el administrador no ha ingresado los gastos del período correspondientes a la comunidad, el sistema bloquea la ejecución del cálculo de gasto común (RF-08) , muestra un mensaje de alerta indicando <i>"No existen gastos registrados para el período actual"</i> y retorna al panel de control financiero.
4a	Propiedad con morosidad previa: Si una propiedad presenta deudas de períodos anteriores, el sistema identifica automáticamente la condición de morosidad y aplica los intereses o recargos configurados según las reglas del sistema (RF-13 y RF-28) , incorporándolos al estado de cuenta actualizado de la propiedad.

4.2. VISTA LÓGICA**4.2.1. Propósito**

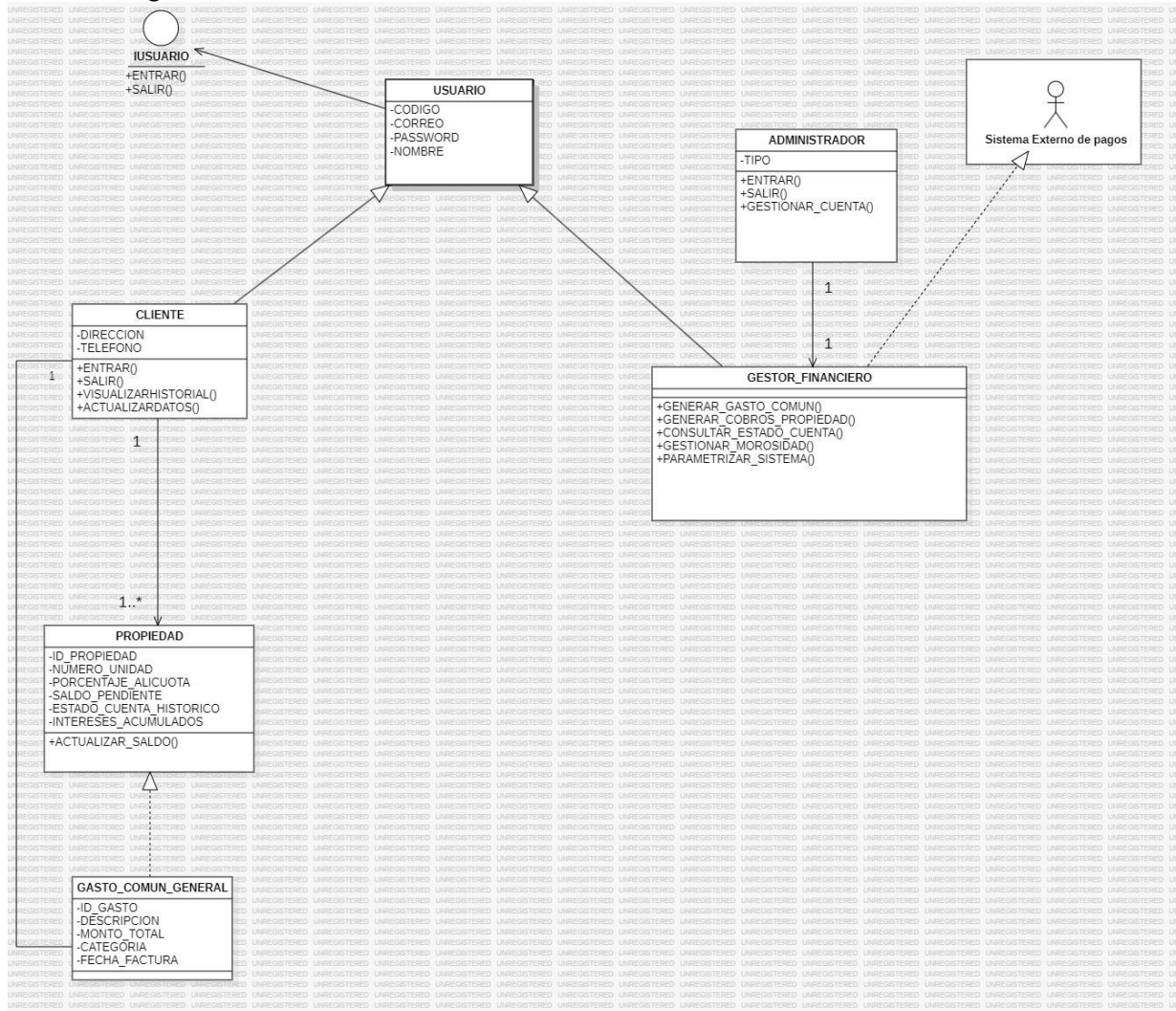
La **vista lógica** describe la estructura interna del **Subsistema Financiero**, mostrando las clases principales del sistema, sus atributos, métodos y las relaciones entre ellas para implementar las reglas de negocio.

Esta vista permite representar cómo el sistema gestiona procesos como la **generación del gasto común del período (RF-08)**, el **prorrateo del gasto común por propiedad (RF-09)**, la **consulta y generación del estado de cuenta por propiedad (RF-12)**, la **identificación de morosidad y cálculo de intereses por**

atraso (RF-13) y la parametrización de las reglas de cobro e intereses del sistema (RF-28).

De esta forma se define la organización lógica de las clases que permiten implementar las funcionalidades financieras del sistema de administración de comunidades.

4.2.2. Diagrama de clases



4.2.3. Descripción diagrama de clases

Se aplica herencia, donde la clase base **USUARIO** contiene los datos generales del sistema. De ella derivan **PROPIETARIO/RESIDENTE** y **ADMINISTRADOR**, quienes interactúan con el sistema para consultar estados de cuenta o gestionar los procesos financieros. Ambos utilizan la interfaz

IUSUARIO para realizar acciones como autenticarse y acceder a la plataforma.

Gestor Financiero (GESTORFINANCIERO):

Es el componente central del subsistema. Se encarga de ejecutar la lógica de negocio relacionada con la generación del gasto común del período (RF-08), el prorrateo por propiedad (RF-09), la identificación de morosidad y cálculo de intereses (RF-13) y la aplicación de las reglas de cobro configuradas (RF-28).

Gestión de Estados de Cuenta (ESTADOCUENTA):

Representa el registro financiero asociado a cada propiedad, donde se almacenan los cargos, pagos y saldos pendientes. Permite la consulta y generación del estado de cuenta individual (RF-12) para cada propietario o residente.

Sistema Externo de Pagos:

Representa la integración con plataformas externas de pago electrónico, encargadas de procesar y validar las transacciones financieras realizadas por los residentes, permitiendo registrar los pagos y actualizar los estados de cuenta dentro del sistema.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

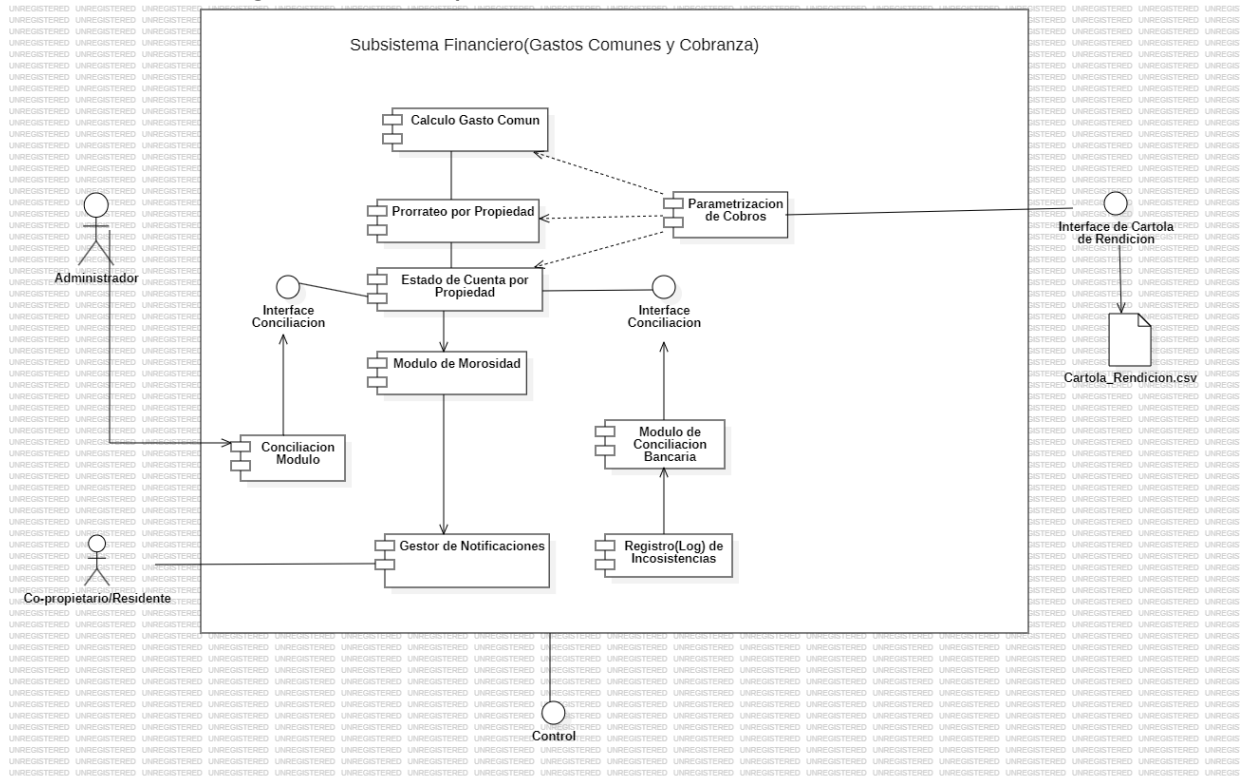
4.3.1. Propósito

El propósito del diagrama de componentes es representar los módulos de software que conforman el Subsistema Financiero y las relaciones o dependencias entre ellos.

Este diagrama permite visualizar cómo se organizan los componentes que implementan funcionalidades como la **generación del gasto común (RF-08)**, el **prorrateo por propiedad (RF-09)**, la **consulta de estados de cuenta (RF-12)**, la **gestión de morosidad e intereses (RF-13)** y la **parametrización de reglas de cobro (RF-28)**.

De esta manera se muestra cómo interactúan los distintos componentes del sistema, como la **interfaz web**, los **controladores financieros**, la **lógica de negocio y el acceso a datos**, garantizando una arquitectura modular y desacoplada.

4.3.2. Diagrama de componente



4.3.3. Descripción diagrama de componente

El diagrama de componentes del **Subsistema 3 (Gestión Financiera: Gastos Comunes y Cobranza)** muestra cómo se organizan los módulos encargados de administrar los procesos financieros del sistema.

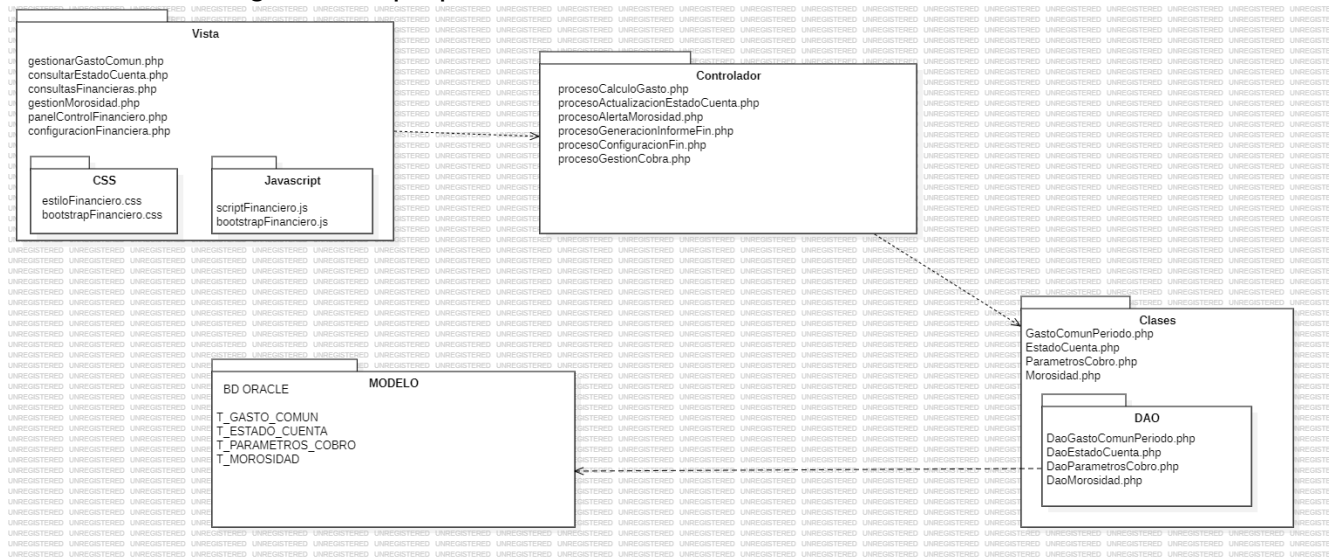
El componente **Cálculo de Gasto Común** permite generar el gasto común del edificio (RF-08), el cual luego es distribuido mediante el componente **Prorratio por Propiedad** (RF-09).

Posteriormente, el componente **Estado de Cuenta por Propiedad** permite visualizar la información financiera de cada unidad (RF-12).

Además, el sistema incluye el **Módulo de Morosidad**, encargado de gestionar deudas e intereses asociados a pagos atrasados (RF-13), considerando las reglas definidas en **Parametrización de Cobros** (RF-28).

El sistema también incorpora un **Módulo de Conciliación Bancaria**, que permite validar los pagos registrados, junto con un **Registro de Inconsistencias** para almacenar posibles errores o diferencias detectadas. Finalmente, el **Gestor de Notificaciones** permite informar a los usuarios sobre estados de cuenta, pagos o situaciones de morosidad.

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

El diagrama de paquetes del Subsistema 3 organiza el sistema usando el patrón MVC (Modelo, Vista y Controlador).

- **Vista:** interfaz en PHP donde los usuarios consultan gastos comunes y estado de cuenta.
- **Controlador:** procesa las acciones del usuario, como generar gastos comunes, calcular prorratesos y detectar morosidad.
- **Clases y DAO:** contienen la lógica del sistema y gestionan el acceso a la base de datos.
- **Modelo (Base de datos):** almacena la información de gastos, cuentas y reglas de cobro en Oracle.

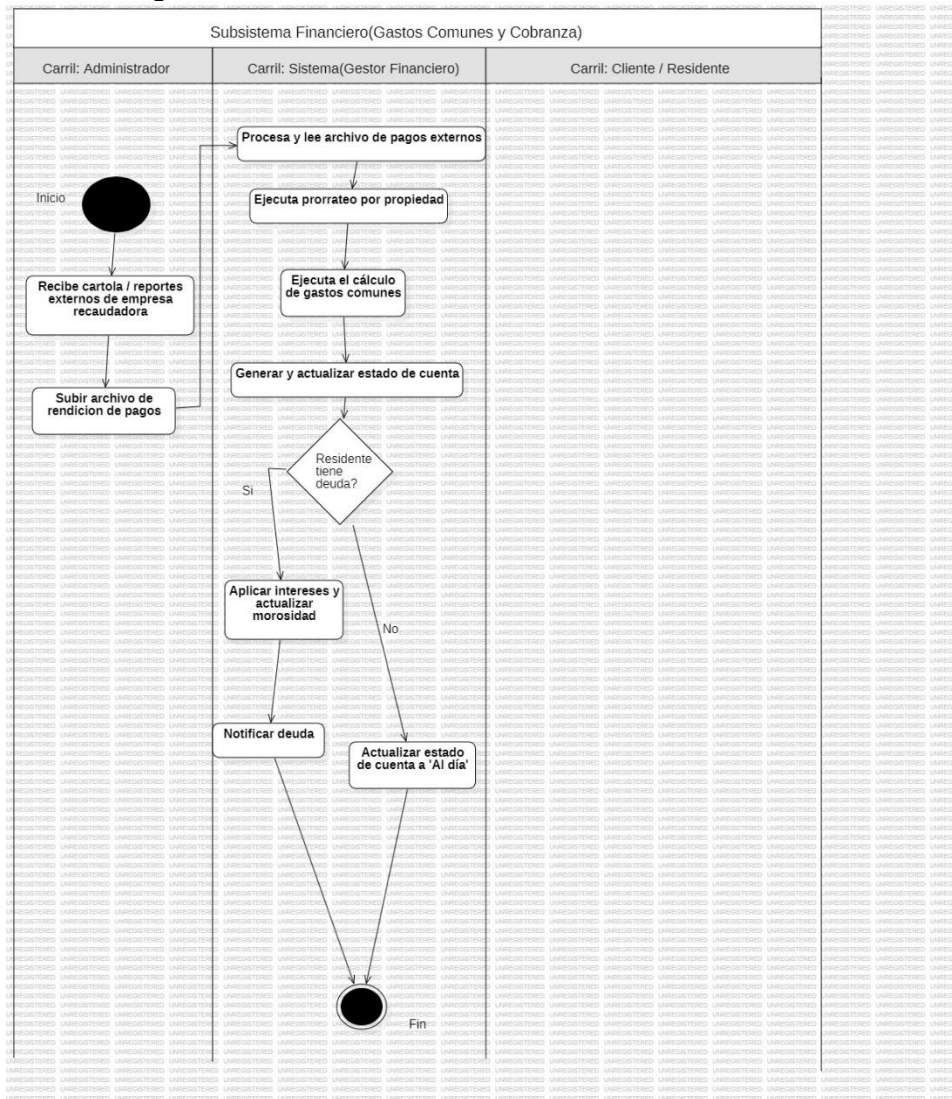
4.4. VISTA DE PROCESOS

4.4.1. Propósito

El propósito de esta vista es representar el flujo de trabajo dinámico del **Subsistema 3: Gestión Financiera (Gastos Comunes y Cobranza)**, mostrando cómo se ejecutan los procesos asociados a la generación de gastos comunes, prorrateso por propiedad, consulta de estados de cuenta y gestión de morosidad.

Permite visualizar los pasos que sigue el sistema desde el cálculo del gasto común del período, su distribución entre las propiedades, la generación de estados de cuenta y la aplicación de reglas de cobro e intereses

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

El diagrama de actividad del **Subsistema Financiero (Gastos Comunes y Cobranza)** representa el flujo del proceso de cálculo y gestión de gastos comunes mediante tres carriles de responsabilidad:

- **Administrador:** recibe y revisa los reportes externos de la empresa recaudadora y carga el archivo de rendición de pagos en el sistema.
- **Sistema (Gestor Financiero):** procesa el archivo de pagos, ejecuta el cálculo del gasto común del período (RF-08), realiza el prorrateo por propiedad (RF-09) y genera o actualiza el estado de cuenta de cada residente (RF-12). Además, verifica si existen deudas y, en caso afirmativo, aplica intereses y registra la morosidad según las reglas configuradas (RF-13 y RF-28).
- **Cliente / Residente:** recibe la notificación del estado de su cuenta actualizado, pudiendo quedar **al día o con deuda registrada** en el sistema.

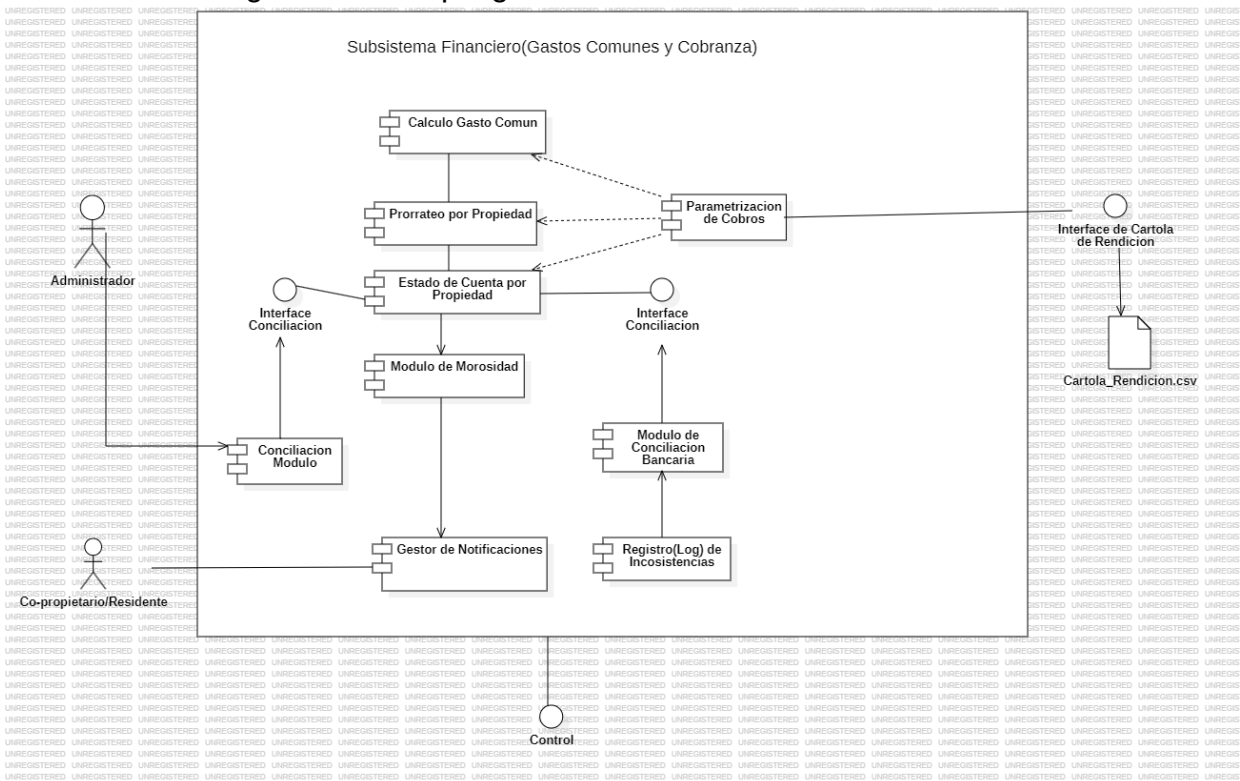
De esta forma, el diagrama muestra cómo el sistema procesa la información financiera y actualiza los estados de cuenta de las propiedades dentro del proceso de gestión de gastos comunes y cobranza.

4.5.VISTA FÍSICA

4.5.1. Propósito

El propósito de este diagrama es representar la arquitectura física y la infraestructura tecnológica del sistema. Detalla la topología de la red mediante la distribución de los componentes de software en servidores independientes: un servidor web encargado de ejecutar la lógica en PHP (MVC) y un servidor aislado que aloja la base de datos Oracle (DAO). Además, describe el uso de protocolos seguros de comunicación como HTTPS y TCP/IP para la conexión con el usuario y la pasarela Webpay, garantizando un entorno escalable, eficiente y protegido contra vulnerabilidades.

4.5.2. Diagrama de despliegue



4.5.3. Descripción diagrama de despliegue

Paso 1: Recopilar la información: El sistema lee un archivo externo con los datos bancarios para saber exactamente qué reglas y montos debe aplicar en los cobros del mes.

- **Paso 2: Calcular y dividir:** Con esa información, el software calcula el gasto común total, lo divide matemáticamente por cada departamento (prorratio) y genera el estado de cuenta final para cada propiedad.
- **Paso 3: Revisar que el dinero cuadre (Conciliación):** El Administrador usa el sistema para validar los pagos de forma manual. Al mismo tiempo, el software hace una revisión automática con el banco; si encuentra alguna diferencia o error, lo deja anotado en un registro para que no se pierda.
- **Paso 4: Avisar a los residentes:** Finalmente, el sistema revisa los estados de cuenta para detectar quiénes están atrasados en sus pagos (morosidad) y se encarga de enviarle automáticamente una notificación de cobro al Co-propietario o Residente.

5. REQUISITOS DE CALIDAD

5.1. Propósito

El propósito de esta sección es definir los atributos de calidad que deben cumplir las funciones del **Subsistema 3: Gestión de Gastos Comunes y Cobranza**. Estos atributos aseguran que los procesos de cálculo del gasto común ,prorratio por propiedad , generación de estados de cuenta, detección de morosidad e intereses y parametrización de reglas de cobro se ejecuten de forma correcta, segura y eficiente.

De esta manera, el sistema garantiza precisión en los cálculos financieros, confiabilidad en la información de cada propiedad y transparencia en la administración de los gastos comunes del edificio.

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN
Usabilidad	Capacidad del sistema para permitir que administradores y residentes consulten estados de cuenta, gastos comunes y pagos de manera clara e intuitiva.	El subsistema debe facilitar la consulta de estados de cuenta por propiedad (RF-12) y permitir comprender fácilmente los cobros, intereses o deudas asociadas a cada departamento.
Rendimiento	Capacidad del sistema para ejecutar procesos financieros y responder a las consultas de los usuarios en tiempos adecuados.	Los procesos de generación de gasto común (RF-08) y prorratio por propiedad (RF-09) deben ejecutarse de forma eficiente para entregar resultados rápidos y mantener actualizado el estado financiero del edificio.
Fiabilidad	Capacidad del sistema para ejecutar cálculos financieros de forma precisa y consistente.	El sistema debe garantizar exactitud en el cálculo de prorratios, estados de cuenta y morosidad (RF-09,

		RF-12, RF-13) evitando errores en los montos cobrados a los residentes.
Seguridad	Capacidad del sistema para proteger la información financiera y restringir el acceso según el rol del usuario.	El subsistema gestiona información sensible como deudas, pagos y reglas de cobro , por lo que debe asegurar el acceso solo a usuarios autorizados y proteger los datos almacenados.
Mantenibilidad	Facilidad para modificar o actualizar reglas de cobro, intereses o cálculos financieros sin afectar el funcionamiento general del sistema.	La parametrización de reglas de cobro e intereses (RF-28) permite adaptar el sistema a cambios administrativos sin necesidad de rediseñar la aplicación.

5.3. Reglas y criterios de evaluación de calidad

Para comprobar de manera objetiva que el Subsistema 3 (Gestión de Gastos Comunes y Cobranza) cumple con los atributos de calidad definidos, se aplicarán las siguientes reglas, métricas y métodos de evaluación sobre los procesos de generación de gastos, prorrateo, consulta de estados de cuenta y gestión de morosidad.

Métricas y Criterios de Aceptación:

• Rendimiento:

El sistema debe ejecutar el proceso de generación del gasto común del período (RF-08) y el prorrateo por propiedad (RF-09) en un tiempo máximo de 5 segundos bajo condiciones normales. Además, la consulta del estado de cuenta por propiedad (RF-12) por parte de los residentes no debe superar 2 segundos de tiempo de respuesta.

• Usabilidad:

Los usuarios deben poder consultar su estado de cuenta, visualizar deudas y entender los cargos aplicados sin dificultad. En las pruebas de uso, al menos el 90% de los usuarios debe ser capaz de consultar su estado de cuenta o identificar su deuda sin errores de navegación.

• Fiabilidad:

El sistema debe garantizar precisión en todos los cálculos financieros relacionados con el prorrateo de gastos comunes (RF-09) y el cálculo de morosidad e intereses (RF-13). La tolerancia al error matemático será 0%, asegurando que los montos calculados coincidan exactamente con los valores definidos en las reglas del sistema.

• Seguridad:

El acceso a la información financiera debe estar controlado por roles de usuario. Los residentes solo podrán consultar su propio estado de cuenta (RF-12), mientras que las funciones de configuración de cobros e intereses (RF-28) estarán restringidas únicamente al administrador del sistema.

- **Pruebas de Carga y Rendimiento:**

Se utilizarán herramientas de simulación de carga para evaluar el comportamiento del sistema cuando múltiples residentes consulten simultáneamente sus estados de cuenta o cuando el administrador ejecute procesos masivos como la generación del gasto común.

- **Pruebas de Usabilidad:**

Se realizarán evaluaciones de la interfaz del sistema con usuarios representativos (administradores y residentes) para verificar que la consulta de gastos comunes y estados de cuenta sea clara y comprensible.

- **Pruebas Unitarias del Cálculo Financiero:**

Se implementarán pruebas automatizadas para validar que los algoritmos de **prorratio de gastos comunes (RF-09)** y **cálculo de intereses por morosidad (RF-13)** generen resultados correctos según las reglas configuradas.

- **Validación de Reglas de Cobro:**

Se verificará que la **parametrización de reglas de cobro e intereses (RF-28)** se aplique correctamente en los cálculos financieros y en la generación de los estados de cuenta.

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt